

Reason → Bucket Reference Table — Card, UPI, E-mandate

Grounding sources:

Card: <https://razorpay.com/docs/errors/payments/cards/> (fetched directly, complete)

UPI: <https://razorpay.com/docs/errors/payments/upi/> (fetched directly — see structural caveat in the UPI section below)

E-mandate: user-provided PDF ("Error Reasons for EMandate" + "Error Reasons for Subsequent Payments"), cross-checked against <https://razorpay.com/docs/payments/recurring-payments/emandate/errors/>

How to read this table

Phase: `registration` | `subsequent` | `both` (one of the 12 shared reasons) — corresponds to the `payment_phase` field, e-mandate only

Bucket: which of your Stage 2 diagnosis buckets this reason maps to, based on reading Razorpay's explanation/next-step text. This is *our* inference on top of Razorpay's docs — Razorpay does not publish a reason→bucket mapping directly.

Hard Decline (Stage 0) candidate: `Y` = permanently unrecoverable, should exit at Stage 0 without reaching Stage 2 at all. `N` = worth letting through to diagnosis/recovery.

Buckets used: `genuine_decline`, `bank_outage`, `fraud_fp`, and `uncertain` — a fourth, first-class Stage 2 output (not a fallback/error state) for reasons that can't be resolved to one of the other three from reason-code alone. See "The `uncertain` bucket" section below for how this is handled, not just labeled.

Full transaction schema — fields the pipeline actually reads

Field	Populated for	Notes
<code>method</code>	all	<code>card</code> <code>upi</code> <code>emandate</code> — flat, no branching
<code>payment_phase</code>	e-mandate only	<code>registration</code> <code>subsequent</code> ; <code>null</code> for Card and UPI — they have no two-phase lifecycle
<code>error_source</code>	all	who: customer / business / internal / gateway / issuer_bank / beneficiary_bank / network / bank (varies by method)
<code>error_step</code>	all	where: method-specific step vocabulary (5 for Card, 15 for UPI Intent, 3 for E-mandate — <code>payment_initiation</code> / <code>payment_authentication</code> / <code>payment_authorization</code> , confirmed directly from Razorpay's docs, same coarse shape as Netbanking)
<code>error_reason</code>	all	why: validated against the reason→bucket tables below, and against <code>payment_phase</code> for e-mandate specifically

Field	Populated for	Notes
join key	UPI, E-mandate only	<code>vpa</code> (UPI) / <code>bank</code> (E-mandate) — used for the row-level Downtime API correlation; Card has no equivalent, since it doesn't route through the outage-join logic

CARD (16 confirmed reasons)

<code>error_reason</code>	Bucket	Hard Decline candidate	Why
<code>authentication_failed</code>	<code>genuine_decline</code>	N	Wrong OTP/closed browser — canonical smart-retry scenario
<code>insufficient_funds</code>	<code>genuine_decline</code>	N	Smart-retry candidate
<code>incorrect_cvv</code>	<code>genuine_decline</code>	N	Simple input correction
<code>payment_cancelled</code>	<code>genuine_decline</code>	N	Explicit customer action
<code>card_not_enrolled</code>	<code>genuine_decline</code>	Y	Needs bank-side enablement first
<code>card_disabled_for_online_payments</code>	<code>genuine_decline</code>	Y	Identical description to <code>card_not_enrolled</code> — likely same underlying scenario, different string
<code>debit_instrument_inactive</code>	<code>genuine_decline</code>	Y	Same description again — third string, same root scenario
<code>debit_instrument_blocked</code>	<code>genuine_decline</code>	Y	Needs bank contact to unblock
<code>card_expired</code>	<code>genuine_decline</code>	Y	Needs new card
<code>transaction_limit_exceeded</code>	<code>genuine_decline</code>	Y	Needs different card/method
<code>bank_technical_error</code>	<code>bank_outage</code>	N	Explicit Downtime API recommendation in the doc
<code>gateway_technical_error</code>	<code>bank_outage</code>	N	Description text identical to <code>bank_technical_error</code> 's — looks like a doc copy-paste, not a real gateway-vs-bank distinction

<code>error_reason</code>	Bucket	Hard Decline candidate	Why
<code>payment_risk_check_failed</code>	fraud_fp	N	Bank declined citing fraud — only fraud-labeled reason in the whole Card list
<code>card_declined</code>	uncertain	N	Razorpay's own text: no access to specific failure detail
<code>payment_failed</code>	uncertain	N	Identical description/next-steps to <code>card_declined</code> — same opacity, different string

Redundancy note: 5 of the 15 distinct-description reasons are really 2 underlying scenarios wearing multiple names (`card_not_enrolled` / `card_disabled_for_online_payments` / `debit_instrument_inactive` share one description; `card_declined` / `payment_failed` share another). Worth collapsing/deduplicating in the generator's sampling logic rather than treating all 5 as independently meaningful.

UPI INTENT (10 confirmed reasons)

Structural caveat: the source page mixes lowercase `reason`-style strings with Title Case headings (e.g. "Customer Bank Account Mismatch," "Partner Bank Downtime," "Partner Bank Technical Issues") nested under them with shared description text. These are most likely `code` field values, not additional `reason` values — but this is our interpretation, not confirmed. Only the 10 snake_case strings below are treated as confirmed `reason` values; verify directly if you want this airtight.

error_reason	Bucket	Hard Decline candidate	Why
insufficient_funds	genuine_decline	N	Smart-retry candidate
payment_cancelled	genuine_decline	N	Explicit customer action
invalid_vpa	genuine_decline	Y	Registration incomplete, needs external action before retry helps
bank_technical_error	bank_outage	N	Explicit Downtime API recommendation
gateway_technical_error	bank_outage	N	Route-side technical fault
credit_failed	bank_outage	N	Maps to the <code>payment_credit_request</code> / <code>response</code> steps — beneficiary-side leg
payment_collect_request_expired	uncertain	N	Customer didn't act within 10 min — abandonment or stuck-waiting, can't tell which
payment_declined	uncertain	N	Vague — "funds could not be debited," no further detail
payment_timed_out	uncertain	N	Same ambiguity as Card's version
vpa_resolution_failed	uncertain	N	Razorpay's own next step is "raise a ticket" — even they don't have a clean answer

No `fraud_fp` reason exists for UPI in this page — confirmed against the full page, not a partial read. Open design decision: should Stage 2 even offer `fraud_fp` as a possible output for UPI transactions, or is that bucket structurally Card/E-mandate-registration-only in this system? Decide explicitly rather than leaving it implicit.

E-MANDATE

Shared across both phases (12)

error_reason	Bucket	Hard Decline candidate	Why
bank_account_invalid	genuine_decline	Y	Account closed/invalid — no retry fixes this without customer re-registering
bank_account_validation_failed	uncertain	N	Could be genuine bad input or third-party validator being flaky — can't tell from

error_reason	Bucket	Hard Decline candidate	Why
			reason alone
bank_technical_error	bank_outage	N	Explicit Core Banking System technical failure — Downtime API territory
debit_instrument_blocked	genuine_decline	Y	Blocked by bank/customer — needs a different method, retry won't help
debit_instrument_inactive	genuine_decline	Y	Same shape as above
gateway_technical_error	bank_outage	N	Gateway-side technical fault — route outage territory
insufficient_funds	genuine_decline	N	Your smart-retry candidate — often resolves on its own shortly after
payment_cancelled	genuine_decline	N	Explicit customer action — could legitimately retry
payment_failed	uncertain	N	Razorpay's own text: "exact reason not communicated to Razorpay" — genuinely opaque
payment_timed_out	uncertain	N	Could be customer inaction or a slow bank/gateway response
server_error	bank_outage	N	Razorpay's own server-side fault
transaction_limit_exceeded	genuine_decline	Y	Structural account limit — retry won't help until customer changes it

Registration-only (14)

error_reason	Bucket	Hard Decline candidate	Why
already_declined	genuine_decline	Y (with 24h retry window)	NPCI-enforced dedup block — not a normal retry case, has its own cooldown rule
authentication_failed	genuine_decline	N	Wrong card/login details — customer can retry immediately
card_expired	genuine_decline	Y	Needs a new card, unrecoverable as-is

error_reason	Bucket	Hard Decline candidate	Why
card_number_invalid	genuine_decline	Y	Bad input, needs correction
duplicate_request	uncertain	N	Gateway-side dedup block — more a business/integration signal than a customer failure; flag <code>source</code> when building this row
incorrect_card_expiry_date	genuine_decline	N	Simple input correction, easy retry
incorrect_cvv	genuine_decline	N	Same
incorrect_otp	genuine_decline	N	Same — this is the canonical "Watch Out" retry scenario from your first screenshot
incorrect_pin	genuine_decline	N	Same
joint_account_not_allowed	genuine_decline	Y	Structural — needs a different account entirely
otp_attempts_exceeded	genuine_decline	Y (temporary block)	Retry-blocked at the bank level for a period, not a normal immediate retry
payment_pending_approval	uncertain	N	Not actually a failure — a pending state; shouldn't be diagnosed as a decline at all if caught mid-flight
payment_risk_check_failed	fraud_fp	N	Only reason in the entire e-mandate schema mapping to this bucket — registration-phase only
user_not_registered_for_netbanking	genuine_decline	Y	Customer needs to complete bank-side registration first

Subsequent-payment-only (6)

error_reason	Bucket	Hard Decline candidate	Why
incorrect_ifsc	genuine_decline	Y	IFSC no longer valid — needs re-registration
input_validation_failed	uncertain	N	Explicitly your/integration-side ("check your integration") — a business-source issue, not really a customer decline

<code>error_reason</code>	Bucket	Hard Decline candidate	Why
<code>invalid_amount</code>	uncertain	N	Same — your-side request malformation
<code>mandate_not_active</code>	genuine_decline	Y	Mandate permanently cancelled — re-registration required, no automated recovery makes sense
<code>payment_declined</code>	uncertain	N	Same opacity as <code>payment_failed</code>
<code>payment_mandate_not_active</code>	bank_outage	N	⚠ Easily confused with <code>mandate_not_active</code> above but means the opposite — bank hasn't finished activating yet, resolves on its own with a retry

The `uncertain` bucket — how it's actually handled, not just labeled

8 of the 32 reasons land here: `bank_account_validation_failed`, `payment_failed`, `payment_timed_out` (shared); `duplicate_request`, `payment_pending_approval` (registration); `input_validation_failed`, `invalid_amount`, `payment_declined` (subsequent). This is not a dumping ground — the point is that when Stage 2 hits one of these, it doesn't force a guess into `genuine_decline` / `bank_outage` / `fraud_fp`. It says so explicitly, with a reason, and the pipeline still does something useful with that:

Stage 1 gets first crack at resolving it. Before Stage 2 settles on `uncertain`, check for cluster corroboration — e.g., a single `payment_timed_out` is uncertain, but 40 of them in the same window against the same bank, corroborated by a Downtime API hit, resolves cleanly to `bank_outage`. `uncertain` should really only surface for isolated, non-clustered cases.

Stage 2's output includes a `resolution_pending_on` field alongside `uncertain` — a short statement of what data would resolve it (e.g. "cluster corroboration against Downtime API," "error_description text needed"). This is what turns the label from a shrug into a stated limitation.

Stage 3 defines a narrow, non-committal default action for `uncertain` — e.g. a single delayed retry with no customer-facing claim about root cause — rather than taking no action at all.

Stage 4 (guardrails) watches the `uncertain` rate. A spike well above the ~25% baseline this table predicts is itself a signal something upstream is off (bad data, a new failure mode not covered by this mapping) — not just normal ambiguity.

Stage 5 (audit trail) logs every `uncertain` classification with the reason, whether cluster context resolved it, and what would have resolved it. This is the demonstrable version of "we preserve ambiguity instead of laundering it" — provable in the audit log, not just claimed in the pitch.

Cross-method takeaways

`fraud_fp` is a thin bucket across the board, not just for e-mandate. Card has exactly one fraud-labeled reason (`payment_risk_check_failed`). E-mandate has it too, but registration-phase only. UPI has none found at all. If a judge asks "how often does your fraud bucket actually fire," the honest answer is: rarely, and its

presence/absence is method- and phase-dependent — worth stating as a deliberate, grounded finding rather than something you'd want to paper over with synthetic balance.

`payment_risk_check_failed` **should never legally appear when** `method == upi`, **or when** `method == emandate` **and** `payment_phase == subsequent`. Both are schema violations worth asserting against in tests/generator validation.

Near-identical reason strings with different meanings show up in more than one method. E-mandate has `mandate_not_active` vs `payment_mandate_not_active` (opposite meanings). Card has 5 reasons collapsing into 2 real scenarios via copy-pasted descriptions. Worth a deliberate normalization step in the generator rather than sampling every string as independently meaningful.

Several reasons across methods (E-mandate's `duplicate_request`/`input_validation_failed`/`invalid_amount`) aren't customer-vs-bank failures at all — they're integration/business-side errors. Decide explicitly whether your generator should model these at all, given the project scope is customer-facing payment degradation.

`uncertain` **shows up meaningfully in all three methods**, not just e-mandate: Card (2 of 15), UPI (4 of 10 — the largest proportion of any method), E-mandate (8 of 32). That's not a modeling weakness — it's the expected shape given your core architectural argument, and UPI having the highest proportion is itself worth mentioning: it's the method where Stage 2's LLM-plus-cluster-context reasoning is doing the most work, precisely because reason-code lookup resolves the least here. The `uncertain` bucket mechanism (Stage 1 cluster-check → Stage 2 states what's missing → Stage 3 safe default → Stage 4 rate monitoring → Stage 5 audit log) applies identically across all three methods, not just e-mandate.